
Gain standard: PLC coding guidelines



Document name: Gain standard v0.2 - PLC coding guidelines
Status: Final

Released by: Gain Automation Technology B.V.
hereafter named Gain

Author: Wino Bode
Last modified by: Wino Bode
Date: 01-05-2022

Table of content

1.	INTRODUCTION	3
1.1.	REVISION HISTORY	3
1.2.	HISTORY AND CONTEXT	3
2.	GAIN PLC STANDARD	4
3.	PLC CODING GUIDELINES.....	5
3.1.	REFERENCE DOCUMENTS	5
4.	NAMING RULES.....	5
4.1.	PREFIXES (N2, N10).....	5
5.	COMMENTING RULES.....	5
6.	CODING PRACTICE RULES	6
6.1.	GENERAL RULES ON CODING PRACTICE	6
6.2.	PREFERRED DATA TYPES (CP22).....	6
6.3.	DATA TYPE CONVERSION (CP25)	6
6.4.	COMPARISON (CP8, CP28).....	6
7.	LANGUAGE RULES (L1, L4, L11)	7
7.1.	GENERAL FORMATTING	7
7.1.1.	<i>Use 4 spaces for default indentation</i>	<i>7</i>
7.1.2.	<i>The maximum line length shall be set to 120 characters</i>	<i>7</i>
7.1.3.	<i>Put a single space character around infix operators (like + and *) and assignments ':= ' except for instance calls.....</i>	<i>7</i>
7.1.4.	<i>For non-keyword unary operators put no space between operand and operator.</i>	<i>7</i>
7.1.5.	<i>No space after opening '(' or before closing parenthesis ')'.....</i>	<i>7</i>
7.1.6.	<i>One space before '(' and one after ')' in expressions</i>	<i>7</i>
7.1.7.	<i>No space before '(' and after ')' in calls.....</i>	<i>7</i>
7.1.8.	<i>No space before semi-colon ';'</i>	<i>8</i>
7.1.9.	<i>At declarations: One space after colon ':'. No tab after.....</i>	<i>8</i>
7.1.10.	<i>No white-space characters at the end of a line.</i>	<i>8</i>
7.1.11.	<i>Always place a space after a comma ',' that does not end a line.</i>	<i>8</i>
7.1.12.	<i>In an array there is no space before and after the brackets []</i>	<i>8</i>
7.1.13.	<i>When line length is not exceeded put THEN/DO etc. on the same line as IF/WHILE....</i>	<i>8</i>
7.1.14.	<i>Use 'AND' and not '&'</i>	<i>8</i>
7.1.15.	<i>Use 'TRUE' and 'FALSE' not 0 and 1 for Boolean values.</i>	<i>8</i>
7.1.16.	<i>In parameter lists keep the parameter types, at least per group, aligned in the same column.</i>	<i>8</i>
7.2.	MULTILINE STATEMENTS.....	9

1. Introduction

1.1. Revision history

Version	Date	Status	Comment	Initials
0.0	02-12-2022	Concept		WBo
0.1	05-12-2022	Draft	A space is always needed around ':='	WBo
0.2	13-01-2023	Draft	Beter explanation of referencing to PLC open. Add examples at chapter 6.3. and 7.1	WBo

Table 1: Revision history

1.2. History and context

These are the coding guidelines for PLC projects. This document can be used by all engineers using the GBL.

It is a 'living' document. Remarks on this document will be appreciated to extend the knowledge which is placed in this document.

GAIN

Automation
Technology

2.

Gain PLC standard

This document is part of the “Gain PLC software standard”. This standard is separated in several documents, each document describes a part of the standard. These documents may be used individually and combined with other documents to create a customer specific standard.

A complete PLC software standard should (at least) contain the following topics:

- PLC naming convention
- PLC software structure
- PLC coding convention
- PLC version control

In this document the “PLC coding guidelines” are described.

GAIN

Automation
Technology

3. PLC coding guidelines

PLC software consists of code, comments, programmable objects, and variable declarations. This section describes the naming conventions for the programmable objects (POU's) and the variable declarations.

This document is an extension of and refers to the PLCopen coding guidelines. The numbering, for example 'N2' and 'CP25', refers to chapters / rules at the PLCopen coding guidelines.

3.1. Reference documents

- plcopen_coding_guidelines_version_1.0.pdf ([Guidelines | PLCopen](#))
- Gain standard - PLC naming conventions.docx – version 2.1

4. Naming Rules

4.1. Prefixes (N2, N10)

By following these naming rules, all objects and variables can be identified via their prefix to know its type and scope.

Therefore, no object or variable instance should be declared without identifier prefix.

See the Gain standard - PLC naming conventions.docx.

5. Commenting Rules

- Comments shall describe the intention of the code (C1)
- All elements shall be commented (C2)
- Use single lines comments in code, multiline commenting is only allowed in formal parameter lists (C5)

Example:

```
(* Multiline
Comments
*)
VAR
    s_nValue1    : DINT;
END_VAR

// Single line comments
s_nValue1 := s_nValue1 + 1;
```

- The default language for comments shall be English (C6)

6. Coding Practice Rules

6.1. General rules on coding practice

- All code shall be used in the application, no dead code is allowed. (CP2)
- Unused variables are also not allowed (CP25)
- Applications shall be well designed, ideally before development phase is started. The design principles shall include object oriented principles like modularization and encapsulation (CP5)
- Use of global variables shall be limited (CP18)
- When using global variables, qualify the global variable list used
- Usage of jump and return should be avoided (CP19)
- Usage of actions should be avoided
- Usage of binary selection operator SEL should avoided

6.2. Preferred data types (CP22)

Preferred datatypes used are DINT for integers and LREAL for floating point values.

6.3. Data type conversion (CP25)

Avoid calculation and rounding errors by explicitly converting data types, preferably use 'overloaded' conversion only defining the target data type.

Example:

```
VAR
  s_nValue1   : DINT;
  s_fValue2   : LREAL;
  s_nResult   : DINT;
END_VAR

// Acceptable when LREAL must be rounded, example: 6 = 2 * 2.5
s_nResult := s_nValue1 * TO_DINT(s_fValue2);

// Even better, example: 5 = 2 * 2.5
s_nResult := TO_DINT(TO_LREAL(s_nValue1) * s_fValue2);
```

6.4. Comparison (CP8, CP28)

Floating point values, Time and physical measurements comparisons shall not be equality or inequality.

These comparisons must only use the following operators: Less than(<), less than or equal(<=), greater than (>), greater than or equal (>=) .

Example:

```
IF s_tonDelay.ET >= T#5S THEN
  // Execute code
END_IF
```

7. Language Rules (L1, L4, L11)

The preferred IEC 61131-3 programming language used is Structured Text.

7.1. General Formatting

7.1.1. Use 4 spaces for default indentation

4 spaces is a single tab.

Example:

```
IF s_tonDelay.ET >= T#5S THEN
    s_nValue1 := 20; // Execute code
END_IF
```

7.1.2. The maximum line length shall be set to 120 characters

The maximum number of characters at a single line is 120.

7.1.3. Put a single space character around infix operators (like + and *) and assignments ':= ' except for instance calls

Example:

```
s_nValue1 := 50 + 20; // Correct code
s_fbFunction1(i_bReqStart := TRUE); // Correct code and preferred.
s_fbFunction2(i_bReqStart:=TRUE); // Correct code but not preferred.
```

7.1.4. For non-keyword unary operators put no space between operand and operator.

The '-' in '-2' is a unary operator.

Example:

```
s_nValue1 := 50 - -20; // Correct code
```

7.1.5. No space after opening '(' or before closing parenthesis ')'

Example:

```
s_nValue1 := (50 + 20); // Correct code
```

7.1.6. One space before '(' and one after ')' in expressions

Example:

```
s_nValue1 := 30 * (50 + 20); // Correct code
s_bStaState1 := (s_nValue1 <= 30) OR (s_nValue1 >= 50); // Correct code
```

7.1.7. No space before '(' and after ')' in calls

Example:

```
s_fbFunction1(i_bReqStart := TRUE); // Correct code
```

7.1.8. No space before semi-colon ';'

Example:

```
s_nValue1 := (50 + 20); // Correct code
```

7.1.9. At declarations: One space after colon ':' . No tab after

```
VAR_INPUT  
    i_bReqEnable : BOOL;  
VAR_END
```

7.1.10. No white-space characters at the end of a line.**7.1.11. Always place a space after a comma ',' that does not end a line.**

Example:

```
s_fbFunction1(i_bReqStart := TRUE, i_bReqStop := FALSE); // Correct code
```

7.1.12. In an array there is no space before and after the brackets []

Example:

```
VAR  
    s_astList_Build : ARRAY[1..10] OF ST_AlarmManager_ListEntry;  
VAR_END  
s_astList_Build[2].sPath := 'Text';
```

7.1.13. When line length is not exceeded put THEN/DO etc. on the same line as IF/WHILE

Example:

```
IF s_tonDelay.ET >= T#5S THEN  
    s_nValue1 := 20; // Execute code  
END_IF
```

7.1.14. Use 'AND' and not '&'

Example:

```
s_bStaState1 := (s_nValue1 >= 30) AND (s_nValue1 <= 50); // Correct code
```

7.1.15. Use 'TRUE' and 'FALSE' not 0 and 1 for Boolean values.

Example:

```
s_fbFunction1(i_bReqStart := TRUE); // Correct code  
s_fbFunction2(i_bReqStart := FALSE); // Correct code
```

7.1.16. In parameter lists keep the parameter types, at least per group, aligned in the same column.

7.2. Multiline statements

When an expression needs to be put on multiple lines start the following line with the operator, and for IF/FOR finishing with THEN/DO on a new line without indentation

Example:

```
IF s_bStaDeviceReady
  AND s_bStaPressureOk
  AND NOT s_bStaMaintenanceRequired
THEN
  _m_RunCode();
END IF
```

When calculations, function calls and function block calls require multiple lines, each line shall have only one operand, starting each newline with its operator, located in the same column.

Example:

```
s_nReturnValue := (s_nInput1
                  + s_nInput2)
                  * c_nByteOffset;
```